

EJERCICIO PRÁCTICO: useState EN FORMULARIOS



OBJETIVO

Aprender a usar `useState` para manejar formularios simples en React.



EJERCICIO 1: MI PRIMER FORMULARIO (30 minutos)

Crea un componente llamado `Saludo.jsx`

El formulario debe tener:

- Un input para el nombre
- Un botón "Saludar"
- Un párrafo que muestre el saludo

Código inicial:

```
jsx
import { useState } from 'react';

function Saludo() {
  // TODO: Crear estado para el nombre

  return (
    <div>
      <h2>¡Vamos a saludarnos!</h2>

      { /* TODO: Input para el nombre */ }
      <input
        type="text"
        placeholder="Escribe tu nombre"
      />

      <button>Saludar</button>

      { /* TODO: Mostrar el saludo aquí */ }
```

```

    <p></p>
  </div>
);
}

export default Saludo;

```

Pasos a seguir:

1. Crear un estado `nombre` que empiece con una cadena vacía
2. Hacer que el input muestre el valor del estado (`value={nombre}`)
3. Crear una función que actualice el estado cuando escribas en el input
4. Al hacer clic en "Saludar", mostrar: "¡Hola [nombre]!"

Pista:

```

jsx
const [nombre, setNombre] = useState("");

```



EJERCICIO 2: FORMULARIO CON DOS CAMPOS (30 minutos)

Crea un componente llamado `Presentacion.jsx`

El formulario debe tener:

- Input para el nombre
- Input para la edad
- Botón "Presentarse"
- Mostrar: "Me llamo [nombre] y tengo [edad] años"

Código inicial:

```

jsx
import { useState } from 'react';

function Presentacion() {
  // TODO: Crear estados para nombre y edad

  return (
    <div>
      <h2>Cuéntanos sobre ti</h2>

```

```

    <div>
      <label>Nombre:</label>
      <input type="text" />
    </div>

    <div>
      <label>Edad:</label>
      <input type="number" />
    </div>

    <button>Presentarse</button>

    { /* TODO: Mostrar la presentación */ }
    <p></p>
  </div>
);
}

```

```
export default Presentacion;
```

Pasos a seguir:

1. Crear dos estados: nombre y edad
2. Conectar cada input con su estado correspondiente
3. Crear funciones para actualizar cada estado
4. Mostrar la presentación cuando se haga clic en el botón

EJERCICIO 3: CONTADOR INTERACTIVO (20 minutos)

Crema un componente llamado `Contador.jsx`

Debe tener:

- Mostrar el número actual
- Botón "+" para sumar
- Botón "-" para restar
- Botón "Reset" para volver a cero

Código inicial:

```
jsx
```

```
import { useState } from 'react';

function Contador() {
  // TODO: Crear estado para el contador

  return (
    <div>
      <h2>Mi Contador</h2>

      { /* TODO: Mostrar el número */ }
      <p>Número: </p>

      <button>+</button>
      <button>-</button>
      <button>Reset</button>
    </div>
  );
}

export default Contador;
```

EJERCICIO 4: LISTA DE TAREAS SIMPLE (40 minutos)

Crea un componente llamado `ListaTareas.jsx`

Debe tener:

- Input para escribir una nueva tarea
- Botón "Añadir"
- Lista que muestre todas las tareas

Código inicial:

```
jsx

import { useState } from 'react';

function ListaTareas() {
  const [nuevaTarea, setNuevaTarea] = useState("");
  const [tareas, setTareas] = useState([]);
```

```

const añadirTarea = () => {
  // TODO: Añadir la nueva tarea a la lista
  // TODO: Limpiar el input
};

return (
  <div>
    <h2>Mis Tareas</h2>

    <input
      type="text"
      value={nuevaTarea}
      onChange={(e) => setNuevaTarea(e.target.value)}
      placeholder="Escribe una tarea"
    />

    <button onClick={añadirTarea}>Añadir</button>

    <ul>
      {/* TODO: Mostrar cada tarea */}
    </ul>
  </div>
);
}

```

```
export default ListaTareas;
```

Pistas:

- Para añadir a un array: `setTareas([...tareas, nuevaTarea])`
- Para mostrar la lista: `tareas.map((tarea, index) => <li key={index}>{tarea}</li>)`



EJERCICIO 5: CAMBIAR COLORES (15 minutos)

Crea un componente llamado `CambiadorColor.jsx`

Debe tener:

- Tres botones: "Rojo", "Verde", "Azul"
- Un div que cambie de color según el botón clickeado

Código inicial:

```
jsx
import { useState } from 'react';

function CambiadorColor() {
  const [color, setColor] = useState('gray');

  return (
    <div>
      <h2>Cambia el color</h2>

      <div
        style={{
          width: '200px',
          height: '200px',
          backgroundColor: color,
          border: '2px solid black'
        }}
      >

      <button onClick={() => setColor('red')}>Rojo</button>
      <button onClick={() => setColor('green')}>Verde</button>
      <button onClick={() => setColor('blue')}>Azul</button>
    </div>
  );
}

export default CambiadorColor;
```



USAR LOS COMPONENTES EN APP.JSX

```
jsx
import Saludo from './components/Saludo';
import Presentacion from './components/Presentacion';
import Contador from './components/Contador';
```

```
import ListaTareas from './components/ListaTareas';
import CambiadorColor from './components/CambiadorColor';

function App() {
  return (
    <div className="App">
      <h1>Ejercicios useState</h1>

      <Saludo />
      <hr />

      <Presentacion />
      <hr />

      <Contador />
      <hr />

      <ListaTareas />
      <hr />

      <CambiadorColor />
    </div>
  );
}

export default App;
```



CONCEPTOS QUE PRACTICAS

- ✨ useState básico
 - ✨ Inputs controlados (value + onChange)
 - ✨ Eventos onClick
 - ✨ Actualizar estados
 - ✨ Mostrar estados en JSX
 - ✨ Arrays en estado (lista de tareas)
-



SI TE ATASCAS

Recordatorio useState:

```
jsx
// 1. Importar
import { useState } from 'react';

// 2. Crear estado
const [miVariable, setMiVariable] = useState(valorInicial);

// 3. Mostrar en JSX
<p>{miVariable}</p>

// 4. Actualizar
setMiVariable(nuevoValor);
```

Input controlado:

```
jsx
<input
  value={miEstado}
  onChange={(e) => setMiEstado(e.target.value)}
/>
```

Botón con función:

```
jsx
<button onClick={() => setContador(contador + 1)}>
  +1
</button>
```